Mastering Multi-Model Vibecoding at Scale: An Architectural & Evaluation White Paper from Inside HackerEarth's VibeCode Arena

Author: Aranya Bahuguna

Platform & Initiative: HackerEarth – VibeCode Arena

Focus: AI-Native Software Engineering, Iterative Specification Chaining, Multi-Model Benchmarking, and Rubric-Driven Code Quality

1. Executive Summary

Software engineering has crossed a critical inflection point. The biggest barrier to application development is no longer the typing of syntactic utterances into an editor – it is the specification, scope, verification, and iteration discipline while directing an autonomous coding agent to avoid subtle architectural debt, edge-case validation gaps, diff inspection debt, and quality assurance liabilities. Within the opportunity provided by HackerEarth on the VibeCode Arena, I have spent hundreds of engineering hours building, auditing, benchmarking, and optimizing production-grade full-stack applications within a live, rubric-scored cloud development environment.

Instead of vague “chatting with AI” about ad-hoc coding requests, HackerEarth’s VibeCode Arena formalizes the concept of “vibecoding” – building software by directing large language models (LLMs) while retaining architectural and validation ownership over the implementation, deployment, diff inspection, and quality assurance. Every project constructed within the Arena is compiled, executed, and objectively scored across five 100-point engineering rubrics:

- Security
- Code Quality
- Correctness
- Performance
- Accessibility

This white paper chronicles the technical methodology, workspace utilization, model orchestration strategies, and domain-specific engineering patterns that enabled me to achieve 100/100 scores on all five rubrics while completing dozens of real-world full-stack code creation challenges inside the Arena.

2. The VibeCode Arena Engineering Environment

Inside the HackerEarth VibeCode Arena (vibecodearena.ai), the developer is presented with a browser-based code construction and evaluation environment. Unlike detached chat interactions with a model which only receive copy-pasted code snippets in response, VibeCode Arena binds a multi-model agentic chat interface to an active workspace with file tree (backend/, frontend/, Makefile), terminal shells, preview servers, git diff history, tool-call telemetry, and an automated evaluation engine.

2.1 Multi-Model Orchestration in a Unified Workspace

A defining capability of the VibeCode Arena is the ability to switch and benchmark between different cutting-edge coding models within a single workspace. Within the Arena, I routinely leverage the following model families:

- OpenAI Family: gpt-5.4, gpt-5.4-mini, gpt-5.4-nano, gpt-5.3-cdex, gpt-5-mini, gpt-5-nano
- Anthropic Family: claude-opus-4, claude-sonnet-4.6, claude-sonnet-4
- Google Family: Gemini 2.5 Pro, Gemini 2.5 Flash, Gemini 3 Flash Preview
- DeepSeek Family: DeepSeek V3 0324, DeepSeek R1
- Qwen, xAI, Mistral Families: Qwen3.5 Flash, Grok 4.5, Mistral Small 3.2 24B

Within the workspace, the code generation context can be restricted to specific file trees (e.g., @backend/ or @Makefile) enabling focused direction of the model.

2.2 Telemetry and Observability: Code, Preview, Metrics, History, and Activity

To observe and debug the natural language construction of deterministic computer programs, VibeCode Arena provides five views that comprise my continuous feedback loop while building applications within the Arena:

Workspace Tab	Core Engineering Function	How I Use It During Active Builds
Code	Full VS Code-style explorer, syntax editor, and dual terminal (Backend Server, Frontend Server)	Monitoring Application bundle generation complete logs, inspecting SQLite/Django/FastAPI migrations, and verifying build zero-warning states
Preview	Live interactive rendering of the compiled frontend talking to the running backend API	Testing interactive states, empty states, filter chips, form validations, and responsive tablet/mobile layouts in real time
Metrics	Automated rubric scorecard grading Security, Code Quality, Correctness, Performance, and Accessibility	Identifying any remaining Blocker, Critical, Major, or Minor defects and driving all five category scores to 100
History	Versioned prompt snapshots (v1, v2, v3, v4…) with Unified and Split git diffs showing exact lines added/removed	Auditing every file patch (e.g., +298 -30 across 4 files) to ensure the model did not accidentally delete existing logic or break CSS layout tokens
Activity	Granular execution trace of every LLM turn and tool invocation (glob, grep, read, apply_patch, bash) with millisecond latency timings	Diagnosing how the agent searches the repository, how many apply_patch turns were attempted, and whether npm run build passed cleanly in bash

3. The Architecture of Chained Iterative Specification

A common anti-pattern when working with coding agents is the use of “monolithic prompting,” attempting to fit an entire multi-page specification document inside a single prompt and hoping that the model will generate a working application from it. In practice, monolithic prompting tends to lead to hallucinated imports, disconnected backend/frontend APIs, and shallow UI states in polyglot applications.

To work effectively inside the VibeCode Arena, I utilize a Chained Iterative Specification architecture. Instead of asking the model to implement an end-to-end specification in a single turn, I decompose it into a series of 10 iterative implementation steps, each of which specifies a narrow aspect of the desired application:

[Prompt 1: Persistence & Schema Foundation] -> [Prompt 2: Core Domain Calculation & Business Rules] -> [Prompt 3: State Transitions, Caps & Guardrail Validations] -> [Prompt 4: Aggregation, Filtering & Supervisor/Audit Queries] -> [Prompt 5: Primary Frontend Dashboard & Visual Hierarchy] -> [Prompt 6: Interactive Creation Forms & Real-Time Input Guards] -> [Prompt 7: Workflow Modals, Handover Actions & Status Toggles] -> [Prompt 8: High-Priority Visual Flags, Overdue Badges & Alerts] -> [Prompt 9: Secondary Views, Timelines, Drawers & Empty States] -> [Prompt 10: Responsive Breakpoints, Touch Targets & Build Polish]

3.1 Why Conversational, Scope-Locked Chaining Works

When executing a single turn inside the VibeCode Arena, an agent's tool loop typically consists of a series of glob and grep calls to explore the file tree, read calls to examine existing code, apply_patch calls to write files, and a bash call to execute build or server commands (e.g., npm run build or backend checks).

By restricting each turn to a focused specification aspect, I ensure:

- The diff footprint is always auditable: Instead of making wholesale changes to 25 files containing 1,500 lines of code, each turn only modifies 2-5 files (e.g., 4 files +298 -30)
- Compiler verification happens at each step: If a TypeScript interface in Angular or Next.js does not match a serializer field in Django or FastAPI, it will be caught at the applicable turn
- Zero context regression: Since each prompt is linked to the previous implementation state ("start by improving the supplier returns view…", "now make the status filters and overdue option cleaner…", "next improve the loading empty and return details states…"), the model retains awareness of the user journey from backend-to-mobile

4. Deep-Dive Technical Case Studies from My VibeCode Arena Portfolio

Within the HackerEarth VibeCode Arena, I have been constructing applications across a variety of operational domains including supply chains, electric and water utilities, field crew safety, and finance. Below are four distinct tech stacks and problem domains from my VibeCode Arena portfolio:

4.1 Case Study 1: Vendor RMA Credit Status Tracker (Angular + Django)

Problem Domain: Within supply chain management, staff dealing with supplier returns under a Return Merchandise Authorization (RMA) often require improved tracking of each return's status throughout its lifecycle, expected versus credited dollar amounts, and identification of overdue credits to vendors.

Technical Implementation & Iterative Refinement: Within the Angular (frontend/src/app/returns/) and Django (backend/rma_tracker/) workspace of the VibeCode Arena, I began the iterative specification refinement by focusing on high-scan, operational list views:

- **List View & Progress Visualization (return-list.component.ts):** Instead of traditional flat text status display, I requested the agent to implement this view with enhanced scanning-friendly status badges and a Credit Progress column containing a progress bar and stage label (0% Not started, 45% Partial, 100% Complete). Overdue returns received special emphasis within the due-date cell to help procurement staff quickly identify delinquent vendors.
- **Compact Filter Bar & Overdue Toggles:** In v3, I requested the agent to modify the status filter bar in the return list view to utilize scanning-friendly icon chips instead of a traditional HTML status dropdown. I further requested a high-contrast overdue toggle pill to show the number of overdue returns.

- **Detail View & Financial Reconciliation (return-detail.component.ts):** In v4, I asked the agent to update the empty, loading, and detail views (4 files +298 -30). The return detail view was specified to contain four summary cards with critical information:
  - RMA status
  - Credit due date
  - Expected credit
  - Credited vs outstanding amounts

  Below the cards, a credit progress block displayed the percentage of recovered funds versus outstanding vendor liability, collapsing responsively across tablet and mobile widths.

  Verification & Result: Each turn concluded with a verification of the code changes in the terminal (Application bundle generation complete [1.139 seconds]) and a final 100/100 score across all five rubrics.

4.2 Case Study 2: Substation & Transformer Inspection History Log (Next.js + FastAPI + SQLite)

Problem Domain: In electrical utility maintenance, field crews conducting regularly-scheduled inspections of electrical substations and transformers frequently encounter recurring hardware faults that are not surfaced to them in a timely manner.

Architectural Design:

- **Relational Persistence Layer (SQLite + FastAPI):** I designed a relational database schema that connected Assets (substation, transformers, region tags, installation dates) to historical Inspection records (inspection date, photo note, condition rating) and any DeferredItems (fault description, asset).
- **Automatic Resurfacing Logic:** The core backend business rule was to always resurface any previously-deferred inspection item from past visits when a new inspection for the same asset was opened.
- **Supervisor Filtering & Chronological Timeline (Next.js):** Supervisors were provided with multi-parameter filter controls (asset_type, region, and an overdue_deferred toggle) along with a complete chronological timeline from asset installation through field photo notes and condition ratings.

4.3 Case Study 3: Utility Budget Billing & Annual True-Up Engine (Vue + Flask)

Problem Domain: For customers on a utility budget billing plan, an annual true-up calculation at the 12-month account anniversary reconciles the customer's actual cumulative usage with the flat monthly payments they have been making. The true-up amount is then utilized to determine the adjusted flat payment for the following year.

Mathematical Precision & Edge-Case Handling:

- **Trailing 12-Month vs. Area Average Initialization:** For established accounts, the true-up calculation utilized the trailing 12-month usage to determine the flat payment, while newly-setup accounts used a region-specific area average.

- **Strict Cent-Rounding Invariant:** Because floating point variance (int(), math.floor()) can accumulate across thousands of accounts, the true-up calculation always rounded to the nearest cent using round(variance, 2) or Decimal quantization before applying it to the next 12-month rate.
- **Customer Running Variance Meter (Vue):** To avoid billing surprises at the end of the year, the Vue frontend displayed a customer-facing running variance meter that showed whether a customer was ahead or behind their flat payment schedule, along with a staff-facing anniversary reconciliation modal.

4.4 Case Study 4: Mid-Cycle Tenant Prorated Billing Splitter (Next.js + FastAPI)

Problem Domain: When a tenant moves out of a unit mid-cycle and a new tenant moves in, billing staff must accurately prorate the cycle charge between the outgoing and incoming tenants based on their length of occupancy without double-billing a single day or failing to release a valid invoice.

State & Validation Pipeline:

- **Zero-Overlap Proration Math:** Given a cycle start date Tstart, cycle end date Tend, and handover date Tmove, the FastAPI calculation engine allocated Dtotal = Tend – Tstart days between the two tenants with Dold + Dnew = Dtotal and Cold + Cnew = Ctotal.
- **Guardrails Against Date Gaps and Overlaps:** If staff attempted to save an occupancy record with invalid gap/overlap dates, the API returned a structured validation error, and the Next.js UI displayed a warning banner.
- **Pending Manager Approval Queue:** All calculated prorated splits were held in a pending_approval state until billing managers reviewed the tenant breakdown in a sortable review queue and released the invoice for printing.

5. Priority-Driven Defect Resolution & The 5-Pillar Evaluation Rubric

Within the VibeCode Arena, simply writing code that “looks good on screen” is not sufficient. The platform’s automated evaluation engine parses the code repository, executing security scanners, functional test suites, performance bundlers, and accessibility checkers.

Whenever I have issues in an evaluation run or am iteratively improving an existing codebase, I follow a 4-Tier defect resolution hierarchy:

- **Tier 1 – Blocker & Critical Issues:** Security vulnerabilities (SQL injection vectors, unescaped XSS bindings, hardcoded secrets, missing CORS/CSRF guards), broken build pipelines, or unhandled server crashes (500 Internal Server Error).
- **Tier 2 – Major Issues:** Broken domain logic, off-by-one proration or cap limits (e.g., failing to block the 9th booking when a technician is capped at 8 meter swaps per day), or unlinked API endpoints.
- **Tier 3 – Minor Issues, Performance & Usability:** Unnecessary re-renders, missing loading skeletons, unoptimized bundle chunks, or confusing form validation feedback.
- **Tier 4 – UI/UX Polish & Visual Hierarchy:** Once all functional and rubric defects are addressed, remaining iterations are focused on semantic iconography, status badge contrast, empty-state illustrations, and responsive tablet/mobile reflow.

VIBECODE ARENA EVALUATION SCORECARD

Category	Score	Blocker	Critical	Major	Minor	Info
Security	100	0	0	0	0	0
Code Quality	100	0	0	0	0	0
Correctness	100	0	0	0	0	0
Performance	100	0	0	0	0	0
Accessibility	100	0	0	0	0	0

5.1 Separating Full-Stack Architecture from “UI-Only” Refinement Passes

An important realization while learning the VibeCode Arena was understanding when to restrict the coding agent to only frontend/UI changes. Whenever the backend API and database schema were already stable or the problem description focused on end-user usability, it was best to avoid making risky changes to the backend file tree. In such cases, I utilized a Strict Frontend-Only Boundary prompting strategy, restricting context mentions and file patches to frontend/ components and stylesheets. The focus was on specifying visual hierarchy and scannable metric cards, semantic icons (location pins, clocks, calendars, status pills), empty/loading/error states, and responsive grid collapse behavior.

For example, while developing the Water Utility Shutoff Planning Board (where field crews log shutoffs by street segment and dispatch monitors open entries past their estimated restoration time), ensuring that the UI layer prominently featured high-scan warning badges for overdue outages, active vs. delayed filter tabs, and one-click resolution buttons turned this into an effective emergency operations board.

6. Comparative Observations Across Frontier Coding Models

Since the VibeCode Arena enables real-time model switching within the IDE while observing tool-call latency in the Activity tab, I have noticed significant differences in behavior across model families for full-stack development tasks:

- **High-Speed Iterative UI & Component Refactoring (GPT-5.4 Mini / Gemini 3 Flash Preview / Claude Sonnet 4.6):** When executing iterative 3-to-4 file frontend patches (e.g., transforming a return list table into scannable status chips with progress bars and running npm run build via bash), smaller, faster models executed the full glob -> read -> apply_patch -> bash loop in 50-to-90 second turns with excellent precision and lower token overhead.
- **Complex Multi-Table Schema & Stateful Backend Logic (Claude Opus 4 / GPT-5.4 / DeepSeek R1):** When designing end-to-end multi-table relational workflows (e.g., tracking utility complaint escalations across departments while flagging households with 3 repeat unresolved faults or managing shared utility vehicle attachment checkouts with 24-hour overdue flags), large language models demonstrated superior ability to maintain consistency across SQLite/Django/Express model definitions and frontend TypeScript interfaces.

7. Conclusion

The opportunity provided by HackerEarth within the VibeCode Arena is a glimpse into the future of professional software development. The platform demonstrates that building applications with AI assistance is not about “hitchhiking” on whatever the model generates – but rather about investing in rigorous engineering discipline around iterative specification, focused scoping, diff auditing, and objective validation of security, quality, correctness, performance, and accessibility. By combining these principles with the real-time telemetry and rubric evaluation engine within VibeCode Arena, developers can successfully ship clean, production-grade full-stack applications that consistently score 100/100 on all fronts.